

Universidad Rafael Landívar
Facultad de Ingeniería
Ingeniería en Informática y Sistemas
Manejo e implementación de Archivos
Docente: Ing. Manuel Gustavo Rojas Vásquez



Aplicación de Escritorio con Gestión de Configuración de Usuario

Cholotío Mendoza, Carlos Andrés – 1517925

Quetzaltenango, Septiembre 2026

1. Justificación del Formato de Almacenamiento

Formato Elegido: JSON

Alternativa Descartada: CSV y Texto plano.

Se seleccionó el formato JSON por encima de CSV basándose en los siguientes criterios técnicos:

- **Facilidad de Parseo y Tipos de Datos:** JSON se mapea de forma nativa a los diccionarios de Python. Mantiene intactos los tipos de datos (el tamaño de la fuente se lee como entero int y los booleanos u otros datos complejos se respetan). En contraste, CSV almacena todo como cadenas de texto (string), obligando a un parseo manual y propenso a errores al leer cada variable.
- **Legibilidad y Estructura:** JSON estructura la información mediante pares de clave: valor. Esto permite que el archivo sea autodescriptivo y fácil de modificar por humanos si fuera necesario. En un CSV, dependeríamos de encabezados o del orden estricto de las columnas para saber qué valor pertenece a qué configuración.
- **Robustez ante Corrupción:** Al poseer reglas de sintaxis estrictas (llaves, comillas dobles, comas separadoras), cualquier edición externa que corrompa la estructura hace que el analizador lance explícitamente un `JSONDecodeError`. Un CSV malformado (por ejemplo, con una coma de más en el nombre de usuario) podría leerse como datos desfasados sin arrojar error, corrompiendo la sesión de forma silenciosa.
- **Tamaño:** Aunque JSON añade unos cuantos bytes extra por las llaves y saltos de línea frente al CSV, en archivos de configuración pequeños esta diferencia de almacenamiento es despreciable a cambio de la seguridad estructural obtenida.

2. Implementación de Escritura Segura y Respaldo (.bak)

Para asegurar que un fallo de energía o cierre abrupto de la aplicación no destruya la configuración del usuario, se implementó un flujo de escritura atómica (segura) de tres pasos.

Lógica implementada:

1. **Respaldo:** Se hace una copia directa del archivo actual a config.bak.
2. **Escritura Temporal:** Se escriben los datos nuevos en un archivo llamado config.tmp.
3. **Reemplazo Atómico:** Se utiliza `os.replace` para sobrescribir el original con el archivo temporal. Esto asegura que el archivo final no quede a medio escribir.

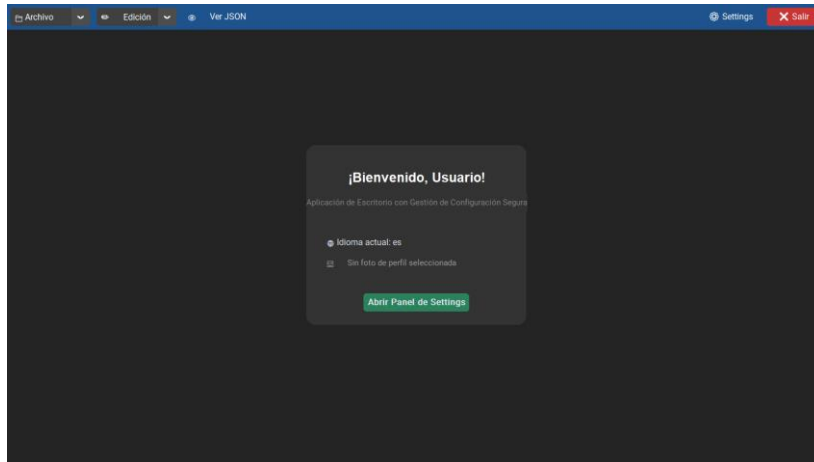
Fragmento de código relevante:

```
def guardar_configuracion(nuevos_datos):  
    try:  
        # 1. Crear respaldo (.bak) del archivo existente  
        if os.path.exists(ARCHIVO_CONFIG):  
            shutil.copy(ARCHIVO_CONFIG, ARCHIVO_RESPALDO)  
  
        # 2. Escribir datos de forma segura en un archivo temporal  
        with open(ARCHIVO_TEMPORAL, 'w', encoding='utf-8') as archivo:  
            json.dump(nuevos_datos, archivo, ensure_ascii=False, indent=4)  
  
        # 3. Reemplazar atómicamente el archivo original  
        os.replace(ARCHIVO_TEMPORAL, ARCHIVO_CONFIG)  
        return True  
    except PermissionError:  
        messagebox.showerror("Error", "Falta de permisos para escribir el archivo.")  
        return False  
  
class Aplicacion(otk.CTk):
```

3. Manejo de Errores Exigidos y Capturas

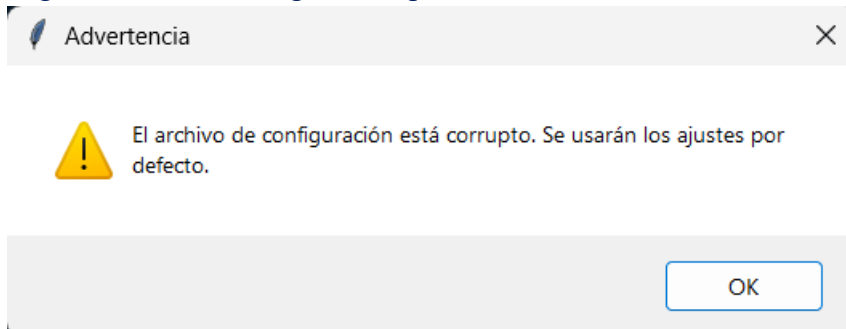
1. Caso A: Archivo Ausente

Cómo se maneja: El código detecta que `os.path.exists()` es falso, no levanta excepciones crudas y procede a cargar el diccionario `CONFIG_POR_DEFECTO`.



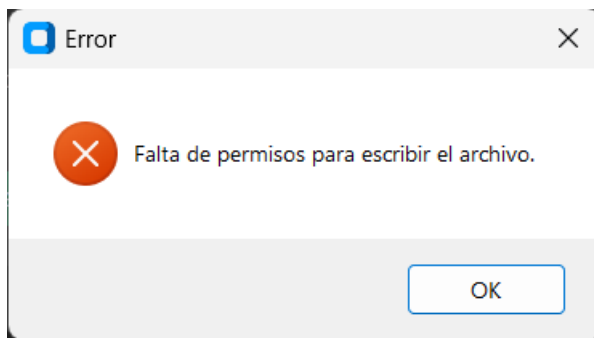
2. Caso B: Archivo Corrupto

Cómo se maneja: Al usar `json.load()`, si el formato falla, atrapamos el `json.JSONDecodeError`, lanzamos una ventana de advertencia al usuario y degradamos a la configuración por defecto.



3. Caso C: Falta de Permisos de Escritura/Lectura

Cómo se maneja: Si el sistema operativo niega el acceso, capturamos el `PermissionError`, avisamos con un mensaje de error y evitamos que el programa se cuelgue o muestre un traceback en consola.



4. Ejemplo del Archivo de Configuración Generado

Para garantizar la correcta codificación de caracteres especiales (tildes, ñ) requeridos en el idioma español, los manejadores de lectura y escritura instancian el parámetro `encoding='utf-8'` explícitamente y `ensure_ascii=False` al hacer el *dump* del JSON.

Ejemplo de config.json generado:

```
1  {
2      "nombre_usuario": "Calush",
3      "tema": "Dark",
4      "idioma": "es",
5      "tamano_fuente": 34,
6      "color_menu": "#ab6ad7",
7      "color_letra": "#ffffff",
8      "foto_perfil": "C:/Users/carlo/Downloads/goat.jpg"
9  }
```